

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados II

Professor: MSc. Jonathan Henrique Jeremias Souza

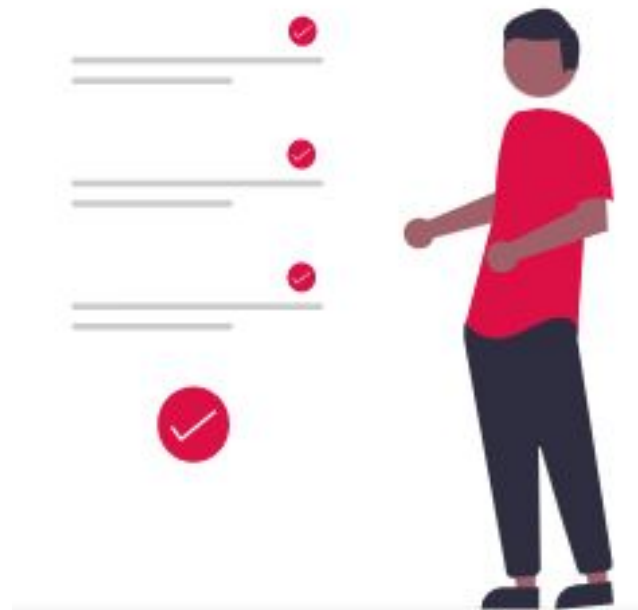
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Classificação
 - Algoritmos de Busca/Pesquisa



Algoritmos de Pesquisa

Introdução

- Certamente já realizou algum tipo de pesquisa, em que tentou encontrar se algum elemento satisfazia a condição desejada.
- Vamos, agora, considerar o seguinte problema: você tem que encontrar um nome em uma lista telefônica.
- Sair folheando as páginas indiscriminadamente seria uma atitude pouco eficaz.

Algoritmos de Pesquisa

Introdução

- De forma instintiva, procuramos o nome abrindo a lista aproximadamente na metade, um pouco mais, ou pouco menos, de acordo com a inicial do nome.
- Se os nomes que aparecem têm suas iniciais maiores que a do nome que estamos procurando, imediatamente dividimos as folhas da esquerda e, caso contrário dividimos as folhas da direita.

Algoritmos de Pesquisa

Introdução

- Essa operação de dividir a lista é realizada até que nos tenhamos aproximado do nome desejado e, assim podemos folhear as páginas para encontrá-lo.
- Esse é um método muito eficaz para realizar uma pesquisa, mas, para isso, **as informações devem estar ordenadas.**

Algoritmos de Pesquisa

Introdução

- Procurar informações em um conjunto desordenado demanda uma operação pouco produtiva, pois deve-se “olhar” cada um dos elementos até encontrar o desejado e, caso ele não exista, você terá consultado todos.
- Pesquisar é uma atividade com a finalidade de encontrar se existe ou não elementos em um conjunto.
- Nesta aula, serão apresentados alguns algoritmos de pesquisa em um conjunto de dados ordenados, quais sejam: pesquisa sequencial, pesquisa binária e transformação de chave.

Algoritmos de Pesquisa

Introdução

- A eficiência de cada uma está na posição em que o dado pesquisado se encontrar, o que poderá ser avaliado após conhecer os algoritmos.

Algoritmos de Pesquisa

Pesquisa

- Pesquisar dados envolve determinar se um valor (denominado chave de pesquisa) está presente no conjunto de dados estudado, e, se estiver, encontrar a localização deste valor.
- A forma mais simples e também a mais onerosa é a **pesquisa exaustiva**.
- Nesta técnica, o algoritmo testa cada elemento até encontrar o desejado e, quando alcança o fim do array, informa ao usuário que a chave não está presente.

Algoritmos de Pesquisa

Pesquisa

- Esta pesquisa é geralmente utilizada em arrays desordenados e a parada do algoritmo ocorre apenas quando o elemento é encontrado ou até que todos sejam pesquisados.
- Quando dispomos de um conjunto de dados ordenados, podemos utilizar algoritmos de pesquisas mais eficientes.
- Além disso, não é necessário percorrer todos os elementos quando o valor não é encontrado, pois normalmente os algoritmos identificam sua ausência e param o processo.

Algoritmos de Pesquisa

Pesquisa sequencial

- Como informado anteriormente, neste algoritmo devemos considerar que o vetor está ordenado.
- A busca se inicia no primeiro elemento e deve testar os elementos sequentes, até encontrar a chave procurada ou até encontrar o primeiro elemento com valor maior que a pesquisada.
- Como o vetor está ordenado, ao encontrar um elemento maior que a chave procurada, não é necessário continuar verificando, pois, todos os elementos seguintes serão maiores.

Algoritmos de Pesquisa

Pesquisa sequencial

- Considere o vetor de dados abaixo e a chave de pesquisa 5. A pesquisa se inicia no primeiro elemento:

1	2	5	7	9	11	12	18	19	21
---	---	---	---	---	----	----	----	----	----

Caso não encontre, é verificado o seguinte:

1	2	5	7	9	11	12	18	19	21
---	---	---	---	---	----	----	----	----	----

Algoritmos de Pesquisa

Pesquisa sequencial

Os elementos seguintes são pesquisados até se encontrar o valor desejado:

1	2	5	7	9	11	12	18	19	21
---	---	---	---	---	----	----	----	----	----

Considere agora a chave de pesquisa 8:

1	2	5	7	9	11	12	18	19	21
---	---	---	---	---	----	----	----	----	----

Algoritmos de Pesquisa

Pesquisa sequencial

- Os elementos serão visitados até se chegar ao valor 9.
- Como ele é maior que a chave de pesquisa (8), o algoritmo é interrompido.
- Temos a certeza de que ele não estará no restante do vetor, pois os demais serão, também, maiores que ele.

Algoritmos de Pesquisa

Implementação da pesquisa sequencial

- O método abaixo retornará o valor da posição do elemento com o valor pesquisado ou a posição de um elemento com o valor maior.
- O usuário deverá verificar se a posição retornada possui o valor pesquisado (encontrou) ou se possui um valor maior que o pesquisado (não encontrou).
- O algoritmo “buscaLinear” recebe (parâmetro) a chave de pesquisa e inicia um comando de repetição (while) com condição de parada: até o final do vetor ou até a chave ser maior que o elemento consultado.

```
1 package busca;
2 public class BuscaLinear {
3
4     public static int buscaLinear(int[] vetor, int num) {
5         int i = 0;
6         while ((i <= vetor.length) && (num > vetor[i])) {
7             i++;
8         }
9         return i;
10    }
11
12    public static void main(String[] args) {
13        int[] vetor = {1, 3, 5, 7, 9};
14        int posicao = buscaLinear(vetor, 5);
15
16        if (posicao < vetor.length && vetor[posicao] == 5) {
17            System.out.println("Encontrado na posição: " + posicao);
18        } else {
19            System.out.println("Não encontrado");
20        }
21    }
22 }
```

Algoritmos de Pesquisa

Pesquisa binária

- Este algoritmo segue a estratégia de dividir para conquistar. Deve-se encontrar o elemento central do array e, a partir daí, compará-lo com a chave de pesquisa.
- Três situações podem ocorrer:
 - o elemento central possui o valor pesquisado, encerra-se o algoritmo;
 - ele é maior que o valor pesquisado e, então, deve-se descartar os elementos a sua esquerda;
 - ele é menor que o valor pesquisado então, devem-se descartar os elementos à sua direita.

Algoritmos de Pesquisa

Pesquisa binária

- Se a chave não foi encontrada, deve-se repetir o processo, com o subconjunto de dados selecionados (esquerda ou direita), até encontrá-la ou o subconjunto ser reduzido até se confirmar que o elemento não existe e, neste caso, deve-se encerrar o algoritmo.
- Considere o vetor de dados abaixo e a chave de pesquisa 5.
- A pesquisa se inicia no elemento central, o qual é encontrado dividindo-se o tamanho do vetor (considerar a soma dos índices: 0 início e 9 fim) por 2.

Algoritmos de Pesquisa

Pesquisa binária

- O resultado deverá considerar apenas a parte inteira. Neste caso temos: $(0+9) / 2 = 4,5$, usar 4:

0	1	2	3	4	5	6	7	8	9
1	2	5	7	9	11	12	18	19	21

Algoritmos de Pesquisa

Pesquisa binária

- Como o valor encontrado é maior que a chave de pesquisa, descartamos a parte direita do vetor e, em seguida, realizamos uma nova divisão.
- Cálculo do meio: $(0+4) / 2 = 2$:

0	1	2	3	4
1	2	5	7	9

E, finalmente, encontramos o valor pesquisado.

Algoritmos de Pesquisa

Implementação Pesquisa binária

- A primeira iteração testa o elemento do meio do array.
 - a) Se isso corresponder a chave de pesquisa, o algoritmo termina.
 - b) Se a chave de pesquisa for menor que o elemento do meio, a chave de pesquisa não poderá localizar nenhum elemento na segunda metade do array e o algoritmo continua apenas na primeira metade.
 - c) Se a chave de pesquisa for maior que o elemento do meio, a chave de pesquisa não poderá localizar nenhum elemento na primeira metade do array e o algoritmo continua apenas com a segunda metade do array.

Algoritmos de Pesquisa

Implementação Pesquisa binária

- Repare que a cada iteração, metade do array é descartada, ficando apenas a metade que interessa.
- E assim sucessivamente até localizar o elemento ou reduzindo o subarray ao tamanho zero.
- O algoritmo retornará um índice (posição de um elemento no vetor) que corresponderá ao valor pesquisado ou um índice com um valor diferente ao pesquisado.
- O usuário deverá realizar a comparação para verificar se foi encontrado.

```
1 package busca;
2
3 public class BuscaBinaria {
4
5     public static int buscaBinaria(int[] vetor, int num) {
6         int meio, inicio, fim;
7
8         inicio = 0;
9         fim = vetor.length - 1;
10        // calcula o elemento central
11        meio = (inicio + fim) / 2;
12
13        while ((inicio < fim) && (num != vetor[meio])) {
14            if (num < vetor[meio]) {
15                fim = meio;
16            } else {
17                inicio = meio + 1;
18            }
19            meio = (inicio + fim) / 2;
20        }
21        if (num == vetor[meio])
22            return meio;
23        else
24            return meio + 1;
25    }
26
27    public static void main(String[] args) {
28        int[] vetor = { 1, 3, 5, 7, 9 };
29        int pos = buscaBinaria(vetor, 6);
30        System.out.println("Resultado: " + pos);
31    }
32 }
```

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- ASCENCIO, A.F.G; ARAÚJO, G. S.; Estruturas de dados: algoritmos, análise da complexidade e implementações em JAVA e C/C++. São Paulo: Pearson, 2010
- PUGA, S.; RISSETTI, G.; Lógica de programação e estruturas de dados com aplicações em Java. 2ª Ed.; Pearson, 2009
- TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J.; Estruturas de Dados Usando C. São Paulo: Pearson, 2009
- Estrutura de dados/ José Marcio Benite Ramos; Liluyoud Cury de Lacerda; Sara Luize Oliveira Duarte; org. Instituto Federal de Educação, Ciência e Tecnologia; Universidade Federal do Mato Grosso - Cuiabá: UFMT; Porto Velho: IFRO, 2013.
- David Menotti: Algoritmos e Estruturas de Dados I - Aula 6 - Recursividade (DECOM – UFOP);